

Context, building, and learning with AI

Context, communication, checking, and building for your learning.

Week 6 · Learn, Vibe, Build · October 5, 2026

What your recent builds taught us

- The AI can finish a prompt and still miss the design you intended.
- A page can look finished while its buttons do not work.
- A local link or a repo alone is not a published website.

Markdown: readable, portable text

- A .md file is a plain-text file—not a special app or database.
- A little punctuation adds structure: headings, lists, links, code.
- Read it in an editor, render it as a page, or paste it into your AI.

Markdown you can read and write

```
# Project notes
## What I want to learn
- Explain commits before changing files.
[Course
lesson](https://cu.learnvibe.build/lessons/w
eek-6/)
Use `git status` to see what changed.
```

Headings, lists, links,
and code—with plain
text underneath.

Context: what the AI can actually see

- Its answer depends on the messages, files and tool results available now.
- It does not automatically know your audience, preferences or project history.
- “Improve my site” is weaker than a clear audience, goal and example.

Save project context instead of repeating it

- Write down what the project is, who it is for, and what should stay true.
- Keep it short, correct and editable; update it when the project changes.
- Example: "For nervous first-time visitors. Plain HTML/CSS. Do not invent event dates."

Automatic loading is tool-specific

- Codex uses AGENTS.md; Claude Code uses CLAUDE.md.
- Other tools have their own rules and supported filenames—check yours.
- In a chat app, paste the context or add it to a Project; it may not read local files.

An index points to deeper context

- A small index says which files exist and what each is for.
- Example: docs/facts.md → verified facts; docs/design.md → visual direction.
- The AI can open the relevant document instead of receiving one giant prompt.

Describe the design, not just the features

- “Make it better” leaves the visual choices to the model.
- Name the audience, feeling, hierarchy and interactions you want.
- Example: “Calm portfolio. Work first, muted colours, clickable project cards.”

Make your feedback observable

- Give a reference or screenshot; say what to keep and what to change.
- Instead of “It’s broken”: “Clicking this card should open the project; nothing happens.”
- Ask for one small change, then compare the result with your intention.

Debugging: find the mismatch

- A bug is a difference between expected and observed behavior.
- Reproduce it: what exact actions cause it? Capture the error or screenshot.
- Ask the AI for a cause and a small fix; rerun the same check afterward.

Check the experience—not just the code

- Open it on a phone. Click every important button and link.
- Try empty input, a strange answer, or a first-time visitor's path.
- For a learning tool, check its facts and quiz answers against a trustworthy source.

When free credits run out

- Save files and a short note: what works, what failed, what to try next.
- Use that note to continue in a fresh session or another tool.
- Reduce the request: one change or one explanation at a time.

Folder, repository, website: different things

- Local folder: files on your own computer.
- Git repository: those files plus a recorded history of versions.
- Deployment: a published version that other people can open.

Commit, push, deploy: three actions

- Commit saves a version and a message in your local Git history.
- Push sends your commits to the repository on GitHub.
- Deploy publishes the project through a host such as GitHub Pages or Vercel.

Your link must work on someone else's phone

- `file:///...` and `localhost:8000` point to your own computer—not a public site.
- A GitHub repository URL shows source; it may not be the live app URL.
- Use the deployed HTTPS URL; test it on another device or in a private window.

A practical path to publishing

- GitHub Desktop: create a repository → commit → publish to GitHub.
- Plain HTML/CSS: GitHub Settings → Pages → main branch / root → Save.
- A framework app may need Vercel: allow repository access, deploy, then test the URL.

API, connector, MCP: what is the difference?

- API: a service's menu of requests that another program can make.
- Connector: a bridge that lets your AI reach a service or files.
- MCP: a shared protocol for AI apps to use tools; an MCP server may call an API.

Access is a choice: read versus write

- Read access can expose data; write access can change or delete it.
- Give only the access needed; review changes before publishing or sending.
- Keep keys, passwords and private data out of public files and repositories.

Use AI to learn about AI

- Ask for an explanation, an example, and a question to check your understanding.
- Say what you already know: “I know HTML; Git is new to me.”
- Compare its explanation with your source—not just how confident it sounds.

The reading: your AI as a coach

- Arielle Shipper · “How to Get Better at AI by Asking AI” · Every
- Ask where you are on the Eight Levels of AI Adoption, with evidence from your work.
- Ask for one next step, then: “Teach me that and help me practise.”

No long chat history? Bring your own evidence

- Paste your last two submissions or describe a recent project.
- Ask: "What can I do already? Show examples. What should I learn next?"
- Treat the AI's assessment as a suggestion; correct its assumptions.

Bring this lesson into your own AI

- Open `cu.learnvibe.build` → Everything for Week 6.
- Click “Copy lesson as markdown”; paste it into Claude or another AI.
- Ask: “Teach me this and quiz me”—then ask follow-ups in your own words.

After the slides: one conversation with Claude

- Aaron will paste the lesson and talk through it with Claude.
- The conversation may lead to building a little thing.
- Watch how questions, context and checking shape the result.

In small groups: build for your learning

- Choose a topic you want to understand or practise, and your own approach.
- Make a quiz, interactive explainer, practice tool—or an idea of your own.
- Use your AI; try the first version yourselves; check it and revise.

How to start—and what to share

- Give your AI the lesson, your learning question, or both.
- Ask for a small first version. Test whether it actually helps you learn.
- Show what you made, what you learned, and one thing you checked or changed.